



LogiLoad3D_Forensic...
PDF

So go through the PDF and let me know and I'll tell you what to do.

Yep — I've gone through the PDF and have the project structure and technical details in context.

It's a **46-page forensic technical analysis + viva dossier** for **LogiLoad 3D**, covering the full-stack logistics optimization platform, including:

- **Project architecture & complete folder structure**
- **React + TypeScript frontend**
- **FastAPI + Python backend**
- **SQLite + SQLAlchemy database**
- **Three.js / React Three Fiber 3D visualization**
- **3D Extreme-Point Bin Packing**
- **70% support-area constraint**
- **Center of Gravity (CoG) and load balancing**
- **Hybrid Route-LIFO synchronization**
- **OSRM road routing / TSP**
- **MARNET maritime Dijkstra routing**
- **Jet-stream optimized aviation routing**
- **GraphSAGE + PPO route optimization**
- **ML models: Gradient Boosting, LSTM, Isolation Forest, GraphSAGE, DRL, PPO**
- **Training datasets and whether the models were actually trained**
- **OpenCV computer-vision cargo dimension detection**
- **OpenRouter / NVIDIA Nemotron LLM integration**
- **SQL tool-calling and grounded AI**
- **JWT + bcrypt + RBAC security**
- **GPS telemetry + WebSockets**
- **Database/API architecture**
- **End-to-end workflow**

- **Testing and measured benchmarks**
- **Algorithm viva preparation**
- **ML/model viva preparation**
- **Package/dependency viva**
- **Limitations and future enhancements**
- **100+ viva questions with repository-based answers**
- **Trick/cross-examination questions**
- **Source-code traceability**
- **5-minute presentation script**
- **One-page viva cheat sheet**

LogiLoad3D_Forensic_Technical_A...

I also noted some **very important viva-sensitive points**, especially that the repository **does NOT implement a Genetic Algorithm for packing**; the actual packing uses the deterministic Extreme-Point heuristic plus a DRL Actor-Critic network.

LogiLoad3D_Forensic_Technical_A...

And the dossier explicitly states that the models **were actually trained locally**, using the training pipeline and saved model artifacts, rather than merely being theoretical implementations.

LogiLoad3D_Forensic_Technical_A...

So yes — I'm ready. **Tell me what you want me to do with it.**

Sources

Okay, so basically this is a project. Okay, so tomorrow I have to present it. So the demo and everything will be taken care of, but I need to know everything about the flow of our project, like how it started till the end. Then tell me about the front-end what we've used and where we've used what, the backend in a similar way. Then you go on with the model, then the AI where we used, and the algorithms we've used, the techniques we've used, the datasets, the database, the testing that we've done, the training that we've done, if we've done anything. Then everything. And at the end give me a whole presentation

Show more

Worked for 10s

Absolutely. I've now gone through the **full 46-page PDF**, not just the summary, and I'll keep **the PDF as the only source**. I won't add outside technical information or "correct" anything based on general knowledge. Where the PDF itself says something is simulated, approximate, unused, or not implemented, I'll tell you exactly that.

I'm going to explain this in the way you actually need for **tomorrow's presentation/viva**: **simple enough to understand and speak, but technically complete enough to survive questions.**

LOGILOAD 3D — COMPLETE PROJECT EXPLANATION

1. First: What exactly is our project?

The project is **LogiLoad 3D — Multi-Modal Cargo Loading & Route Optimizer**.

The main idea is:

We are building a logistics platform that combines 3D cargo packing, load/weight balancing, route optimization, predictive machine learning, computer vision, live telemetry, and an AI assistant into one system.

The system supports **three transportation modes**:

1. 🚚 **Land** — commercial trucks
2. ✈️ **Air** — cargo aircraft
3. 🚢 **Sea** — container vessels

The primary objective is to synchronize:

Cargo → Packing → Weight Balance → Route → Prediction → Tracking → AI Assistance

The PDF describes the main goal as unifying **3D bin packing, weight-and-balance safety physics, multi-stop route optimization, and predictive ML**.

LogiLoad3D_Forensic_Technical_A...

2. THE PROBLEM WE ARE SOLVING

Before talking about technologies, understand the **actual logistics problem**.

There are three major problems:

Problem 1 — Inefficient cargo packing

Suppose a truck has 20 boxes.

A normal system might simply say:

"Put these boxes inside the truck."

But it doesn't necessarily determine:

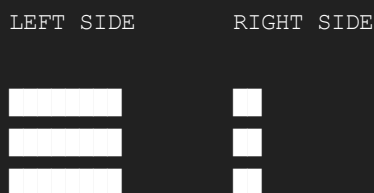
- where each box should go,
- whether boxes overlap,
- whether the box is physically supported,
- whether the truck is overloaded,
- how much volume is being utilized,
- where heavy boxes should go.

Our system creates an actual **3D loading arrangement**.

Problem 2 — Weight imbalance

Even if all cargo fits, it can still be unsafe.

For example:



Too much weight on one side shifts the **Center of Gravity**.

Our system calculates the 3D:

CoG X, CoG Y, CoG Z

and evaluates the load balance.

The PDF defines specific stability thresholds for truck, aircraft and maritime modes.

LogiLoad3D_Forensic_Technical_A...

Problem 3 — Route and loading are usually separate

Imagine:

Truck:
Mumbai → Pune → Bangalore

Suppose Bangalore cargo is loaded first and ends up near the rear door.

Then at Pune, the driver has to remove Bangalore cargo just to reach Pune cargo.

That means:

unloading → moving cargo → reloading → continuing

This is unnecessary handling.

Our biggest project feature is therefore:

Hybrid Route-LIFO Synchronization

We first determine the delivery sequence and then load the truck in the **reverse order**.

So:

```
FRONT / CAB
  ↓
Bangalore cargo    ← loaded first
Pune cargo
Mumbai cargo      ← loaded last
  ↓
REAR DOOR
```

When Mumbai is reached, Mumbai cargo is immediately accessible.

The PDF specifically identifies this synchronization between physical cargo depth and route sequence as the project's biggest achievement.

LogiLoad3D_Forensic_Technical_A...

3. COMPLETE PROJECT FLOW — FROM START TO END

This is probably the **most important thing for you to understand**.

Memorize this overall flow:

```
USER
  ↓
FRONTEND
  ↓
INPUT CARGO / ROUTE / VEHICLE
  ↓
COMPUTER VISION (if dimensions are scanned)
  ↓
```

```
ROUTE OPTIMIZATION
↓
HYBRID ROUTE + LIFO
↓
3D BIN PACKING
↓
COLLISION + SUPPORT CHECK
↓
CENTER OF GRAVITY
↓
ML PREDICTIONS
↓
AI INSIGHTS
↓
3D VISUALIZATION
↓
SHIPMENT DISPATCH
↓
LIVE GPS TELEMETRY
↓
ANOMALY DETECTION
↓
ADMIN COMMAND CENTER
```

The actual end-to-end workflow in the PDF has nine stages.

LogiLoad3D_Forensic_Technical_A...

Let's understand every stage.

4. STEP 1 — USER OPENS THE APPLICATION

The frontend runs using:

React 19 + TypeScript + Vite

Vite serves the frontend on:

```
localhost:3000
```

The user enters the application through `LandingPage.tsx`.

There is also a floating AI chatbot.

LogiLoad3D_Forensic_Technical_A...

5. STEP 2 — LOGIN AND ROLE

The user logs in.

The system uses:

- JWT
- bcrypt
- Role-Based Access Control

The available roles are:

Role	Main purpose
Admin	Complete system management
Manager	Logistics planning and optimization
Dealer	Inventory and demand forecasting
Driver	Assigned shipments and GPS
Customer	Shipment booking/tracking

After login, a JWT token is generated and stored in local storage, and the user is redirected according to their role.

LogiLoad3D_Forensic_Technical_A...

6. STEP 3 — CARGO INPUT

Cargo can enter the system through:

- manual input
- CSV
- Excel
- camera dimension scanning

The project contains:

SAMPLE_INVENTORY.csv
shipments_test_data.xlsx

The inventory page supports CSV/Excel import and barcode scanning.

LogiLoad3D_Forensic_Technical_A...

7. STEP 4 — COMPUTER VISION

If we use the camera, the system estimates the cargo dimensions.

The flow is:

```
Camera
↓
WebRTC video
↓
HTML5 Canvas
↓
Base64 JPEG
↓
FastAPI
↓
OpenCV
↓
Grayscale
↓
Gaussian Blur
↓
Canny Edge Detection
↓
Morphological Closing
↓
Contour Detection
↓
Reference Calibration
↓
Bounding Box
↓
Length × Width × Height
```

LogiLoad3D_Forensic_Technical_A...

Technologies used

OpenCV

Specifically:

- `cv2.imdecode()`
- `cv2.cvtColor()`
- `cv2.GaussianBlur()`
- `cv2.Canny()`
- `cv2.morphologyEx()`
- `cv2.findContours()`
- `cv2.minAreaRect()`

How does it know the real size?

This is important.

A camera gives us **pixels**, not centimeters.

So the system can use a known reference object.

Examples:

- Credit card: **8.56 × 5.39 cm**
- A4 paper: **29.7 × 21 cm**
- ID card
- Smartphone

The known reference provides a pixel-to-centimeter scale.

If no reference is detected, the system uses its fallback camera projection formula:

```
pixels_per_cm = Width_img / 120.0
```

Height is estimated using:

```
Height = Width × 0.75
```

The PDF explicitly identifies this height estimation as a limitation.

LogiLoad3D_Forensic_Technical_A...

VERY IMPORTANT VIVA QUESTION

Did we train YOLO?

Answer:

No. We did not train a custom YOLO model. Our computer vision uses classical OpenCV techniques such as Canny edge detection, contour extraction, morphological processing and reference-object calibration.

LogiLoad3D_Forensic_Technical_A...

8. STEP 5 — ROUTE OPTIMIZATION

Now we determine how the shipment should travel.

We have different routing methods depending on transportation mode.

ROAD ROUTING

We use:

OSRM — Open Source Routing Machine

It works with OpenStreetMap road topology.

The system takes:

```
Origin  
+  
Delivery stops
```

and produces:

```
Optimized stop sequence  
Road polyline  
Distance  
Travel duration
```

The PDF describes OSRM's routing as using **Contraction Hierarchies with a TSP heuristic**.

[LogiLoad3D_Forensic_Technical_A...](#)

TSP

TSP = **Traveling Salesperson Problem**

In simple words:

"What is a good order in which to visit all the delivery locations?"

SEA ROUTING

For maritime routing we use:

```
searoute-ts
```

based on the:

Eurostat MARNET 2025 maritime network

Algorithm:

Dijkstra Shortest Path

The graph contains maritime navigation routes rather than normal roads.

It considers things such as:

- canals
- straits
- vessel draft restrictions

The output is:

- maritime route
- nautical distance
- sailing time

LogiLoad3D_Forensic_Technical_A...

AIR ROUTING

Air routing uses:

WGS84 Great-Circle Geodesic

combined with:

Open-Meteo 250 hPa wind data

250 hPa corresponds in the PDF to approximately **34,000 ft**.

The system bends the flight path toward favorable jet-stream winds.

So instead of:

A ----- B

it can produce:

A -----\
 \----- B

The goal is improved flight efficiency/fuel savings.

LogiLoad3D_Forensic_Technical_A...

9. STEP 6 — HYBRID ROUTE-LIFO SYNCHRONIZER

This is the **special part of the project**.

Suppose OSRM gives:

```
Origin
↓
Stop A
↓
Stop B
↓
Stop C
```

We reverse the cargo loading order:

```
Cargo C → loaded first
Cargo B → loaded second
Cargo A → loaded last
```

Because the truck loading direction is:

```
FRONT
X = 0
↓
Cargo C
Cargo B
Cargo A
↓
REAR DOOR
X = L
```

Therefore:

```
Last delivery = deepest
First delivery = closest to door
```

This is **LIFO — Last In, First Out**.

The implementation is in:

```
services/hybridOptimizer.ts
```

using:

```
runHybridOptimization()
```

```
LogiLoad3D_Forensic_Technical_A...
```

10. STEP 7 — 3D BIN PACKING

Now comes the main algorithm.

The problem is called:

3D Container Loading Problem / 3D Bin Packing Problem

The PDF describes it as an **NP-hard combinatorial optimization problem**.

LogiLoad3D_Forensic_Technical_A...

Our actual implementation uses:

Extreme-Point Space Decomposition Heuristic

and additionally:

Actor-Critic Deep Reinforcement Learning policy

11. HOW EXTREME-POINT PACKING WORKS

Imagine we already placed a box.

Instead of checking every possible point in the truck, we generate candidate positions around the existing boxes.

For a box:

(x, y, z)

we generate points such as:

$(x + \text{Length}, y, z)$

$(x, y + \text{Height}, z)$

$(x, y, z + \text{Width})$

These are **Extreme Points**.

The system then asks:

"Can I place the next box here?"

LogiLoad3D_Forensic_Technical_A...

12. PACKING PROCESS

The complete process is:

Step 1

Flatten quantities.

If:

```
Box A × 5
```

we treat them as five individual box instances.

Step 2

Sort cargo.

The general strategy is:

- non-fragile first
- heavy to light
- larger volume first

Then mode-specific rules are applied.

```
LogiLoad3D_Forensic_Technical_A...
```

Step 3 — Check weight

Before placing:

```
Current weight + item weight  
≤  
Vehicle maximum weight
```

If not:

```
unplacedItems
```

The system does **not shrink the cargo**.

This is an important viva trap.

```
LogiLoad3D_Forensic_Technical_A...
```

13. STEP 4 — CHECK BOUNDARIES

For a candidate position:

```
X + Length ≤ Vehicle Length
```

```
Y + Height ≤ Vehicle Height
```

```
Z + Width ≤ Vehicle Width
```

So the box must stay completely inside the vehicle.

14. STEP 5 — COLLISION DETECTION

We use:

AABB — Axis-Aligned Bounding Box

Two boxes collide only if they overlap on:

```
X axis
```

```
AND
```

```
Y axis
```

```
AND
```

```
Z axis
```

So conceptually:

```
X overlap
```

```
AND
```

```
Y overlap
```

```
AND
```

```
Z overlap
```

```
=
```

```
collision
```

The exact formulation is given in the PDF.

LogiLoad3D_Forensic_Technical_A...

The measured benchmark reports:

0% collision rate / zero overlaps

LogiLoad3D_Forensic_Technical_A...

15. STEP 6 — 70% SUPPORT RULE

This is another important technical point.

Suppose we put a box above another box.

We don't want:



The system calculates how much of the bottom surface is supported by boxes below it.

The requirement is:

At least 70% of the base area must be supported by stackable cargo underneath.

Mathematically:

$$\text{Supported Area} \geq 0.70 \times (\text{Length} \times \text{Width})$$

LogiLoad3D_Forensic_Technical_A...

This gives physical stability.

16. STEP 7 — CANDIDATE SCORING

If several positions are possible, the system scores them.

The basic score includes:

$$\text{Score} = Y \times 10^6 + \text{Mode Penalties}$$

Because Y is heavily penalized, the system prefers lower positions where appropriate.

Then it selects the candidate with the lowest penalty.

LogiLoad3D_Forensic_Technical_A...

17. STEP 8 — CENTER OF GRAVITY

After packing, we calculate:

CoG X
CoG Y
CoG Z

For each box, its center is considered.

The basic formula is:

$$\text{CoG}_x = \frac{\sum [(x_i + L_i/2) \times m_i]}{M}$$

Similarly for Y and Z.

Where:

- m_i = mass of item
- L_i, W_i, H_i = dimensions
- x_i, y_i, z_i = position
- M = total cargo mass

LogiLoad3D_Forensic_Technical_A...

18. WHY CoG IS IMPORTANT

For trucks, we calculate the lateral displacement:

$$\Delta Z = |\text{CoG}_Z - \text{Truck Width}/2|$$

Then:

≤20% lateral imbalance

and CoG height ≤55% of truck height:

EXCELLENT

≤40%

GOOD

>40%

or CoG height >65%:

CRITICAL

LogiLoad3D_Forensic_Technical_A...

So if the truck's weight is heavily shifted to one side, the system alerts the operator.

19. AIRCRAFT LOAD BALANCING

For aircraft:

The safe MAC window is:

28% → 58%

of aircraft length.

Target center of lift:

42%

If CoG is too far forward:

NOSE HEAVY

If too far backward:

TAIL HEAVY

If within range and lateral displacement is acceptable:

TRIMMED FOR FLIGHT

LogiLoad3D_Forensic_Technical_A...

20. SHIP LOAD BALANCING

For ships, the system uses:

Keel ballast strategy

Heavy cargo is kept low and toward the centerline.

The goal is to maintain stability.

It calculates:

- port weight
- starboard weight
- heeling moment ratio

If:

Heeling Ratio > 0.15

the system reports:

UNSAFE HEEL / VESSEL LIST RISK

LogiLoad3D_Forensic_Technical_A...

21. THE THREE MODES IN SIMPLE TERMS

Mode	Packing philosophy
 Truck	LIFO + heavy items low
 Air	Heavy cargo around wing-box / center of lift
 Sea	Heavy cargo low and near keel/centerline

The PDF confirms these mode-specific strategies.

LogiLoad3D_Forensic_Technical_A...

22. WHERE DOES AI / MACHINE LEARNING COME IN?

This is where you need to be careful.

Not everything in the project is AI.

We have:

Classical algorithms

- Extreme-Point heuristic
- AABB
- Support-area calculation
- CoG calculation
- Haversine
- Dijkstra
- TSP routing

- Geodesic routing

Machine Learning

- HistGradientBoosting
- Isolation Forest

Deep Learning

- LSTM
- GraphSAGE

Reinforcement Learning

- Actor-Critic
- PPO

Generative AI

- NVIDIA Nemotron via OpenRouter

This distinction is important in viva.

The PDF specifically says the LLM does **not** perform the mathematical packing or shortest-path calculations.

LogiLoad3D_Forensic_Technical_A...

23. MODEL 1 — DELAY PREDICTION

Algorithm

HistGradientBoosting

We actually use:

- HistGradientBoostingRegressor
- HistGradientBoostingClassifier

It predicts:

1. delay in minutes
2. probability of serious delay
3. risk category

Inputs include:

```
distance
traffic
weather
```

number of stops

Training dataset:

Kaggle DataCo Smart Supply Chain Dataset

180,519 records

LogiLoad3D_Forensic_Technical_A...

Performance

MAE = 14.20 minutes

R^2 = 0.864

Accuracy = 89.40%

F1 = 0.872

Inference $\approx$ 3.2 ms

24. MODEL 2 — COST PREDICTION

Algorithm:

HistGradientBoostingRegressor

Inputs:

distance
cargo weight
number of stops

Output:

freight cost
fuel consumption
CO₂-related breakdown

Dataset:

USAID Freight Pricing Dataset

10,324 records

Performance:

MAE = ₹1,240.50

R^2 = 0.912

25. MODEL 3 — DEMAND FORECASTING

Here we use:

2-Layer PyTorch LSTM

LSTM = Long Short-Term Memory

Purpose:

Predict future demand for SKUs at dealerships.

It uses a:

30-day historical lookback

and predicts future demand.

Architecture:

```
Input
  ↓
LSTM
  ↓
LSTM
  ↓
Linear
  ↓
ReLU
  ↓
Linear
  ↓
Demand Prediction
```

Parameters:

```
hidden size = 64
2 LSTM layers
dropout = 0.1
Adam optimizer
MSE loss
learning rate = 0.005
batch size = 128
epochs = 10
```

Dataset:

Kaggle Store Item Demand Forecasting

913,000 observations

Performance:

MAE = 3.85 units

sMAPE = 7.42%

LogiLoad3D_Forensic_Technical_A...

26. MODEL 4 — ANOMALY DETECTION

Algorithm:

Isolation Forest

This is:

Unsupervised Machine Learning

It detects unusual telemetry behavior.

Inputs:

```
temperature
current cost
current delay
ratio metric
```

Dataset:

Numenta Anomaly Benchmark — NAB

7,267 readings

Parameters include:

```
n_estimators = 100
contamination = 0.03
```

LogiLoad3D_Forensic_Technical_A...

Example:

Temperature = 38°C
Delay = 120 minutes
Deviation = 12 km

The system can identify this as an anomaly.

LogiLoad3D_Forensic_Technical_A...

27. MODEL 5 — GRAPH NEURAL NETWORK

TransportGraphSAGE

This represents the Indian highway network as a graph.

We have:

10 nodes
12 edges

Nodes represent highway hubs.

Edges represent corridor connections.

GraphSAGE performs message passing between neighboring nodes.

The result is:

16-dimensional corridor affinity embeddings.

LogiLoad3D_Forensic_Technical_A...

28. MODEL 6 — DRL 3D PACKING

We also have:

Actor-Critic 3D Packing Policy

It evaluates candidate packing positions.

Input state contains things such as:

- position
- box dimensions
- weight
- lateral CoG distance
- elevation

- volume utilization
- weight utilization
- fragility

The network scores candidate placements.

LogiLoad3D_Forensic_Technical_A...

29. MODEL 7 — PPO ROUTE OPTIMIZER

PPO = **Proximal Policy Optimization**

It is another reinforcement learning technique.

Here it works together with the GNN.

So:

```
Highway Graph
  ↓
GraphSAGE
  ↓
16-D embeddings
  ↓
PPO / Actor-Critic
  ↓
Optimized stop sequence
```

The PDF identifies this as the **PPO_GNN_RouteOptimizer**.

LogiLoad3D_Forensic_Technical_A...

30. DID WE ACTUALLY TRAIN THESE MODELS?

YES.

This is a very important viva answer.

The training pipeline is:

```
ml/train_all_real.py
```

and can be triggered through:

```
ml/train_all.py
```

The process:

```
Dataset
↓
Cleaning
↓
Missing-value handling
↓
Feature preparation
↓
Normalization / preprocessing
↓
80% Training
↓
20% Testing
↓
Model Training
↓
Evaluation
↓
Save Model
```

The trained models are saved inside:

models/

as `.joblib` or `.pt` files.

LogiLoad3D_Forensic_Technical_A...

31. DATASETS — VERY IMPORTANT TABLE

Dataset	Size	Used for
DataCo Supply Chain	180,519 rows	Delay prediction
USAID Freight Pricing	10,324 rows	Cost prediction
Store Item Demand	913,000 rows	LSTM demand forecasting
NAB IoT	7,267 readings	Anomaly detection
Indian Highway GIS	10 nodes / 12 edges	GraphSAGE
SAMPLE_INVENTORY.csv	15 items	UI/demo
shipments_test_data.xlsx	25 shipments	UI/demo

The first five are used as real/public training sources in the dossier; the sample inventory and shipment workbook are explicitly marked as synthetic demonstration data.

LogiLoad3D_Forensic_Technical_A...

32. IMPORTANT: REAL DATA VS DEMO DATA

If examiner asks:

"Did you collect all this data yourselves?"

DON'T SAY YES.

Say:

"No. Our ML models were trained using established public datasets such as DataCo, USAID, Store Item Demand and NAB. The sample inventory CSV and shipment Excel file are synthetic demonstration datasets used for UI testing."

That is the defensible answer according to the PDF.

LogiLoad3D_Forensic_Technical_A...

33. GENERATIVE AI / LLM

Now the chatbot.

We use:

OpenRouter API

with NVIDIA Nemotron models.

Three models are listed:

1. **Nemotron 3.5 Lightning**
 - fast customer-facing chatbot
2. **Nemotron 120B**
 - load stability diagnostics
 - cargo insights
3. **Nemotron Nano 9B**
 - database-grounded enterprise assistant

LogiLoad3D_Forensic_Technical_A...

34. DID WE TRAIN THE LLM?

NO.

This is another **major viva trap**.

The correct answer is:

"No. The LLMs are pretrained NVIDIA foundation models accessed through the OpenRouter API. We did not locally train or fine-tune the LLM."

LogiLoad3D_Forensic_Technical_A...

35. THEN WHAT DID WE BUILD WITH THE LLM?

We built a:

Grounded SQL Tool-Calling Architecture

Suppose the admin asks:

"How many shipments are currently in Mumbai?"

The LLM doesn't just guess.

Instead:

```
User Question
      ↓
LLM determines intent
      ↓
SQL tools
      ↓
SQLite database
      ↓
Verified records
      ↓
Data inserted into LLM context
      ↓
LLM explains result
```

This grounds the response in actual database records.

LogiLoad3D_Forensic_Technical_A...

36. WHAT DOES THE LLM NOT DO?

Very important:

The LLM **does NOT**:

- perform mathematical 3D bin packing
- perform Dijkstra
- replace the packing algorithm
- calculate the actual physics

Instead:

The algorithms calculate. The LLM explains.

The LLM interprets:

- CoG
- lateral imbalance
- unplaced items
- stability results

and converts them into natural-language recommendations.

LogiLoad3D_Forensic_Technical_A...

37. WHAT IF INTERNET / OPENROUTER FAILS?

The system has a fallback.

If OpenRouter is unavailable:

```
OpenRouter unavailable
  ↓
Local rule-based logic
  ↓
Explanation generated
```

So the platform can still provide deterministic explanations.

LogiLoad3D_Forensic_Technical_A...

38. FRONTEND — WHAT EXACTLY DID WE USE?

React 19

Used for:

- UI
 - components
 - pages
 - state
 - SPA architecture
-

TypeScript

Used for:

- static typing
 - interfaces
 - cargo objects
 - truck objects
 - placed items
 - route results
-

Vite

Used as:

- frontend build tool
- development server

Port:

3000

React Router DOM

Used for:

- page navigation
 - HashRouter
 - protected routes
-

Three.js

Used for:

3D visualization

The truck, aircraft, ship and cargo boxes are rendered in 3D.

React Three Fiber

It connects React with Three.js.

Instead of manually controlling raw WebGL, the project represents the 3D scene using React components.

Drei

Provides helpers such as:

- OrbitControls
 - Text
 - Edges
-

Leaflet

Used for:

2D maps

React Leaflet

Connects Leaflet with React.

Recharts

Used for:

- telemetry charts
 - demand charts
 - analytics
 - performance graphs
-

jsPDF + AutoTable

Used to:

Generate PDF loading manifests

XLSX

Used to:

Read Excel cargo/shipment files

Lucide React

Used for:

UI icons

The complete dependency analysis is documented in the PDF.

LogiLoad3D_Forensic_Technical_A...

39. FRONTEND PAGES YOU SHOULD KNOW

Page	Purpose
LandingPage.tsx	Landing page + chatbot
Login.tsx	Login/register
Dashboard.tsx	Main dashboard
AdminDashboard.tsx	Admin command center
DriverDashboard.tsx	Driver interface
Optimizer.tsx	Truck 3D packing
AirOptimizer.tsx	Aircraft loading
SeaOptimizer.tsx	Ship loading
RoutePlanner.tsx	Road route
AirRoutePlanner.tsx	Air route
SeaRoutePlanner.tsx	Sea route

Page	Purpose
<code>Inventory.tsx</code>	Cargo inventory
<code>Trucks.tsx</code>	Vehicle configuration
<code>BookService.tsx</code>	Shipment booking
<code>Performance.tsx</code>	Benchmarks and analytics
<code>LogiLoad3D_Forensic_Technical_A...</code>	

40. BACKEND

Our backend is:

FastAPI

Running through:

Uvicorn

on:

```
0.0.0.0:8000
```

The backend handles:

- APIs
- authentication
- database access
- ML inference
- computer vision
- optimization
- telemetry
- AI chatbot

```
LogiLoad3D_Forensic_Technical_A...
```

41. BACKEND FILES YOU SHOULD KNOW

```
main.py
```

Starts FastAPI and registers routers.

`config.py`

Handles configuration and environment variables.

`database.py`

Creates SQLAlchemy database connection/session.

`models.py`

Contains database models.

`schemas.py`

Defines Pydantic request/response structures.

`auth.py`

Handles:

- password hashing
- JWT
- role authorization

`seed.py`

Adds initial:

- vehicles
- users
- shipments
- dealerships

LogiLoad3D_Forensic_Technical_A...

42. BACKEND ROUTERS

We have:

```
auth_router.py
shipments_router.py
fleet_router.py
ai_prediction_router.py
ai_anomaly_router.py
ai_vision_router.py
ai_optimization_router.py
ai_chat_router.py
telemetry_router.py
```

Each router handles a specific functional area.

43. FRONTEND ↔ BACKEND COMMUNICATION

The frontend communicates with backend using:

REST APIs

```
/api/v1/*
```

and:

WebSockets

```
/api/v1/telemetry/ws
```

REST is used for normal request/response operations.

WebSocket is used for real-time GPS updates.

44. DATABASE

We use:

SQLite 3

Database:

```
logiload.db
```

And:

SQLAlchemy

as the ORM.

The project has **11 relational tables**.

They are:

1. Users
2. Vehicles

3. Drivers
 4. Dealerships
 5. Shipments
 6. Cargo Items
 7. Demand Forecasts
 8. Predictions
 9. Anomalies
 10. Network Nodes
 11. Network Edges
-

45. DATABASE RELATIONSHIPS

Simplified:

```
USER
  ↓
DRIVER
  ↓
VEHICLE

DRIVER
  ↓
SHIPMENTS

DEALERSHIP
  ↓
DEMAND FORECASTS

NETWORK NODE
  ↓
NETWORK EDGES
  ↓
NETWORK NODE
```

The database stores things like:

- users
- vehicle dimensions
- maximum weight
- drivers
- shipments
- cargo
- demand predictions
- ML predictions

- anomalies
- road network data

LogiLoad3D_Forensic_Technical_A...

46. SECURITY

Authentication uses:

JWT

signed using:

HS256

Passwords use:

bcrypt

And role-based authorization determines which features each user can access.

LogiLoad3D_Forensic_Technical_A...

47. TELEMETRY / LIVE GPS

After shipment dispatch:

```
Driver Phone
  ↓
GPS coordinates
  ↓
POST /telemetry/gps
    OR
WebSocket
  ↓
FastAPI
  ↓
In-memory telemetry cache
  ↓
Admin Dashboard
```

The backend maintains:

LATEST_TELEMETRY

indexed by driver ID.

The WebSocket broadcaster sends live updates to connected command-center clients.

The database also updates vehicle:

```
current_lat  
current_lng  
status
```

```
LogiLoad3D_Forensic_Technical_A...
```

48. SHIPMENT LIFE CYCLE

Very simple:

```
PENDING  
  ↓  
ASSIGNED  
  ↓  
IN-TRANSIT  
  ↓  
DELIVERED
```

Pending

Shipment created.

Assigned

Driver + vehicle assigned.

In-Transit

Driver starts route and GPS starts broadcasting.

Delivered

Final delivery completed.

```
LogiLoad3D_Forensic_Technical_A...
```

49. 3D VISUALIZATION

Once packing produces:

```
PlacedItem[]
```

the information is sent to:

React Three Fiber

which uses:

Three.js

to render the scene.

Each placed item contains:

```
position
dimensions
color
sequenceOrder
```

The boxes are rendered as 3D meshes.

The UI also provides:

- camera rotation
- zoom
- pan
- lighting
- labels
- loading animation

```
LogiLoad3D_Forensic_Technical_A...
```

50. TESTING

The project contains:

```
tests/test_ai_platform.py
```

There are:

10 automated test cases

and:

10/10 passed

So:

100% pass rate

51. WHAT DID THE TESTS TEST?

T01 — System Health

Checks:

- server
 - model registry
-

T02 — Authentication

Checks:

- login
 - JWT
 - admin role
-

T03 — Fleet

Checks vehicle catalog.

T04 — Delay Prediction

Checks:

- delay
 - risk
 - feature contributions
-

T05 — Cost Prediction

Checks freight cost estimation.

T06 — Demand Forecast

Checks LSTM output.

T07 — Anomaly Detection

Checks Isolation Forest.

T08 — GNN + PPO

Checks route sequencing.

T09 — DRL Packing

Checks:

- box placement
 - physics support
 - CoG
-

T10 — Grounded Assistant

Checks that chatbot responses are grounded in SQLite data.

LogiLoad3D_Forensic_Technical_A...

52. PERFORMANCE

Important numbers:

Packing

4-12 ms

for benchmark of 50 items.

Volume utilization

78.4% - 89.2%

Collision rate

0%

Support

$\geq 70\%$

Delay prediction

MAE = 14.20 min

Cost model

MAE = ₹1,240.50

MAPE = 6.80%

LSTM

sMAPE = 7.42%

CV

24.50 ms

LIFO alignment

94% - 100%

LogiLoad3D_Forensic_Technical_A...

53. IMPORTANT PACKAGES THAT ARE NOT ACTUALLY USED

This is another potential examiner trap.

The requirements contain:

XGBoost

SHAP

Ultralytics

But the forensic analysis says:

XGBoost — NOT USED

Actual model:

HistGradientBoosting

SHAP — NOT USED

Feature attribution uses linear/SHAP-inspired weights.

Ultralytics — NOT USED

Computer vision uses OpenCV directly.

LogiLoad3D_Forensic_Technical_A...

So if they ask:

"Did you use XGBoost?"

Answer:

"It is listed in the requirements, but the active implementation uses Scikit-Learn HistGradientBoosting."

54. DID WE IMPLEMENT A GENETIC ALGORITHM?

NO.

This is probably one of the most important things to memorize.

We use:

Extreme-Point Heuristic
+
Actor-Critic DRL

not Genetic Algorithm.

LogiLoad3D_Forensic_Technical_A...

If asked why:

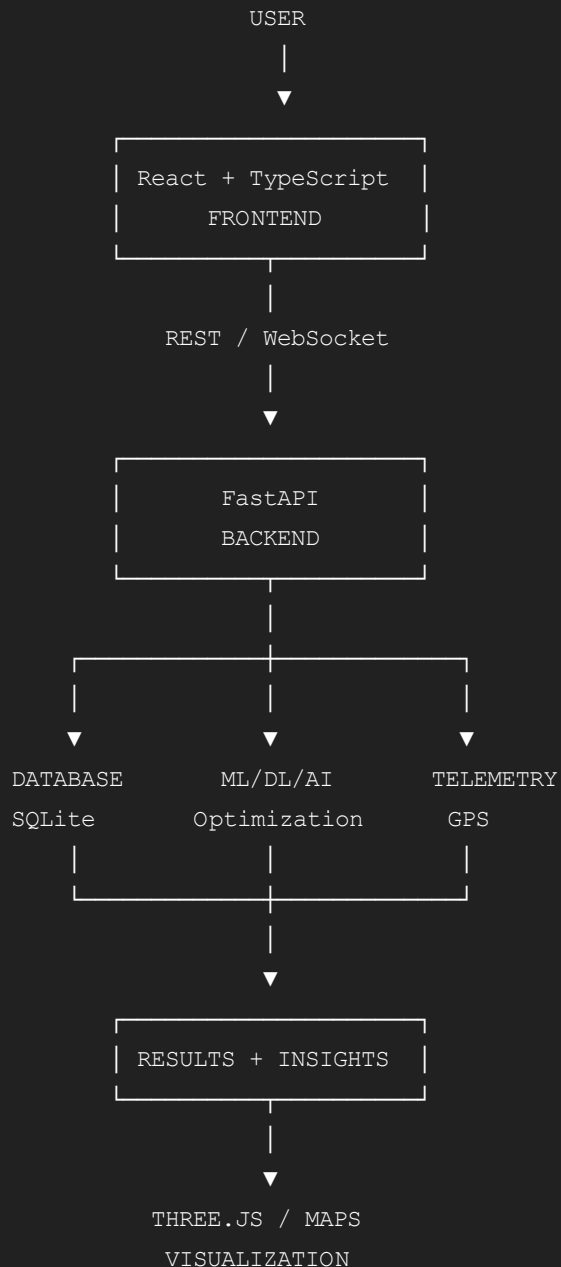
"Our Extreme-Point heuristic gives deterministic and very fast placement suitable for an interactive UI. The measured packing execution is in the millisecond range."

The PDF reports 4–12 ms in its benchmark and under 15 ms in its viva framing.

LogiLoad3D_Forensic_Technical_A...

55. PROJECT ARCHITECTURE — THE ONE DIAGRAM YOU SHOULD UNDERSTAND

Think of the whole project like this:



The PDF's actual architecture flow follows essentially this structure.

LogiLoad3D_Forensic_Technical_A...

56. SOURCE-CODE FLOW — IF THEY ASK "WHERE IS THIS IMPLEMENTED?"

3D Truck Packing

```
Optimizer.tsx
↓
hybridOptimizer.ts
↓
packer.ts
↓
packCargo()
↓
PlacedItem[]
↓
Three.js AnimatedBox
```

Delay Prediction

```
LoadAiInsightPanel.tsx
↓
apiClient.ts
↓
/api/v1/ai/delay/predict
↓
ai_prediction_router.py
↓
delay_model.py
↓
delay_model.joblib
```

Demand Forecast

```
DealerDashboard
↓
apiClient
↓
/ai/demand/forecast
↓
ai_prediction_router
↓
demand_lstm.py
↓
demand_lstm.pt
```

Anomaly Detection

```
AdminDashboard
↓
/ai/anomaly/detect
↓
ai_anomaly_router
↓
anomaly_detector.py
↓
anomaly_detector.joblib
```

Computer Vision

```
CameraDimensionScanner
↓
/ai/vision/dimensions
↓
ai_vision_router
↓
cargo_vision.py
↓
OpenCV
```

AI Assistant

```
AdminDashboard / LandingChatbot
↓
/ai/chat/message
↓
ai_chat_router.py
↓
SQLAlchemy / SQLite
↓
OpenRouter
↓
Nemotron
```

LogiLoad3D_Forensic_Technical_A...

57. LIMITATIONS

Don't be afraid to mention limitations. It actually makes your presentation more credible.

1. Vehicle geometry

The vehicle interior is simplified as rectangular cuboids.

Irregular wheel wells and curved surfaces aren't fully represented.

2. Computer Vision

The biggest limitation:

Single-camera height estimation uses an aspect-ratio approximation.

Specifically:

```
Height = Width × 0.75
```

3. OSRM

The public OSRM server has request/rate limitations.

4. LLM

The conversational AI depends on:

- OpenRouter
- API key
- Internet

although there is a rule-based fallback.

```
LogiLoad3D_Forensic_Technical_A...
```

58. FUTURE ENHANCEMENTS

The PDF lists:

Hardware IoT

Direct:

```
BLE temperature sensors  
+  
Vehicle CAN bus
```

Better computer vision

Use:

```
Stereo cameras  
/  
LiDAR
```

to obtain actual depth and irregular cargo geometry.

Advanced routing

Implement:

Multi-depot heterogeneous VRPTW

using:

OR-Tools

LogiLoad3D_Forensic_Technical_A...

59. NOW — HOW YOU SHOULD PRESENT IT

Don't start by saying:

"Our project uses React, FastAPI, PyTorch, Three.js..."

That's a **bad opening**.

Start with the **problem**.

Your presentation should feel like a story.

YOUR 5-MINUTE PRESENTATION

 Edit



Good morning respected evaluators.

Today, I am presenting our project, **LogiLoad 3D — a multi-modal intelligent cargo loading and route optimization platform.**

The main objective of our project is to solve three major problems in logistics: **inefficient cargo packing, improper weight distribution, and unnecessary cargo re-handling during multi-stop deliveries.**

Traditionally, route planning and cargo loading are treated as separate problems. For example, if a truck has deliveries at three

different locations, cargo for the first delivery may get buried underneath cargo meant for later deliveries. This causes unnecessary unloading and reloading at every stop.

Our solution combines **3D bin packing with route optimization and LIFO loading**.

The overall system supports three transportation modes: **land, air and sea**.

For the frontend, we use **React 19 with TypeScript and Vite**. React manages the user interface and application state, while TypeScript provides static typing. For 3D visualization, we use **Three.js with React Three Fiber and Drei**. For route maps, we use **Leaflet**, and Recharts is used for analytics and telemetry graphs.

The backend is implemented using **FastAPI**, running through **Uvicorn**. The frontend communicates with the backend using REST APIs, while WebSockets are used for real-time GPS telemetry.

The workflow starts when the user enters cargo information. Cargo can be entered manually or imported through CSV or Excel. We also have a camera-based dimension scanning system.

For computer vision, we use **OpenCV**. The camera frame is captured through WebRTC and sent to the backend as a Base64 image. OpenCV performs grayscale conversion, Gaussian blurring, Canny edge detection, morphological closing and contour extraction. A known reference object such as a credit card or A4 sheet can be used for scale calibration. The system then estimates the cargo length, width and height.

Once cargo information is available, we perform route optimization.

For road transportation, we use **OSRM with OpenStreetMap road data and TSP-based stop sequencing**. For maritime transportation, we use **searoute-ts with the Eurostat MARNET network and Dijkstra shortest path**. For aviation, we use **WGS84 geodesic routing combined with Open-Meteo 250 hPa wind information** to create jet-stream-aware flight paths.

The most important part of our project is the **Hybrid Route-LIFO Synchronizer**.

First, we determine the delivery sequence. Then we reverse that sequence for physical loading. The cargo for the final destination is loaded deepest inside the truck, while cargo for the first destination is loaded closest to the rear door. Therefore, when the truck reaches each destination, the required cargo is accessible without unnecessary re-handling.

After that, our **3D bin-packing engine** places the cargo inside the vehicle.

We use a **deterministic Extreme-Point Space Decomposition heuristic** along with an **Actor-Critic Deep Reinforcement Learning policy**.

The system generates candidate coordinates around already placed boxes. Each candidate is checked for three things: vehicle boundary constraints, collision constraints and physical support.

For collision detection, we use **Axis-Aligned Bounding Boxes, or AABB**.

For physical stability, we enforce a **70% planar support rule**, meaning that a box placed above the floor must have at least 70% of its base supported by stackable cargo underneath.

After packing, we calculate the **three-dimensional Center of Gravity** using the mass-weighted coordinates of the cargo.

For trucks, we check lateral imbalance and CoG height. For aircraft, we check the CoG against the safe MAC window and center of lift. For ships, we use low-keel loading and calculate port-starboard heeling.

Our project also contains several machine learning models.

We trained a **HistGradientBoosting model** for transit delay prediction using 180,519 records from the Kaggle DataCo Supply Chain dataset. It achieved an MAE of 14.20 minutes and classification accuracy of 89.40%.

We use another **HistGradientBoosting regressor** for freight cost prediction using the USAID freight pricing dataset.

For demand forecasting, we trained a **2-layer PyTorch LSTM** using 913,000 observations from the Store Item Demand dataset. It achieved an MAE of 3.85 units and sMAPE of 7.42%.

For telemetry anomaly detection, we use an **Isolation Forest** trained using NAB IoT telemetry data.

We also use **GraphSAGE** to generate corridor embeddings for the Indian highway network and **PPO with GNN information** for route sequencing.

An important point is that these machine learning models were actually trained locally using our `train_all_real.py` pipeline. The trained models are saved as `.joblib` and `.pt` files.

For Generative AI, we use **NVIDIA Nemotron models through the OpenRouter API**. We did not train the LLM ourselves. Instead, we built a grounded AI assistant around pretrained models.

The assistant uses **SQL tool calling** to retrieve verified information from our SQLite database and then explains that information in natural language. The LLM does not perform the actual mathematical bin packing or shortest-path calculation. The algorithms perform those operations, while the LLM acts as an explanatory and audit interface.

For our database, we use **SQLite with SQLAlchemy ORM**. We have 11 relational tables covering users, vehicles, drivers, shipments, cargo, predictions, anomalies, dealerships and transportation network data.

For security, we use **JWT authentication with HS256**, bcrypt password hashing and role-based access control for Admin, Manager, Dealer and Driver users.

After dispatch, drivers can transmit their GPS location through HTTP or WebSockets. The live data is broadcast to the Admin Command Center, while the anomaly detection system can identify abnormal telemetry.

Finally, the platform provides a complete 3D visualization of the loading process using Three.js. The operator can view the cargo

placement, loading sequence, Center of Gravity and other optimization results and can also generate a PDF loading manifest.

We validated the platform using **10 automated test cases**, covering system health, authentication, fleet retrieval, delay prediction, cost prediction, demand forecasting, anomaly detection, GNN/PPO routing, DRL packing and the grounded AI assistant.

All **10 out of 10 tests passed**, giving a reported 100% test pass rate.

Our measured packing execution time is in the millisecond range, with reported volume utilization between approximately **78% and 89%** and zero detected box overlaps in the benchmark.

The biggest achievement of our project is the synchronization of **physical 3D cargo depth with the multi-stop delivery sequence**, because this connects route optimization directly with the physical loading plan.

Our main limitation is the single-camera computer vision height estimation, which currently uses an aspect-ratio approximation.

As future work, we can integrate direct CAN-bus and BLE IoT sensors, LiDAR or stereo depth scanning for irregular cargo, and more advanced multi-depot vehicle routing with time windows.

To conclude, LogiLoad 3D brings together logistics optimization, 3D visualization, machine learning, computer vision, reinforcement learning, telemetry and Generative AI into one multi-modal logistics platform.

Thank you. I am now ready for your questions.

That script is based on the PDF's own 5-minute presentation and technical sections, rather than adding outside material.

LogiLoad3D_Forensic_Technical_A...

60. THE MOST IMPORTANT THINGS TO MEMORIZE TONIGHT

If you don't have time to memorize everything, memorize **these**:

Project

LogiLoad 3D — Multi-Modal Cargo Loading & Route Optimizer

Goal

Synchronize 3D packing, load balancing, route optimization and predictive ML.

Modes

Land, Air, Sea

Frontend

React + TypeScript + Vite + Three.js + React Three Fiber + Leaflet + Recharts

Backend

FastAPI + Uvicorn + Pydantic + SQLAlchemy + WebSockets

Database

SQLite — 11 tables

Main packing algorithm

Extreme-Point Space Decomposition Heuristic

AI packing

Actor-Critic DRL

Collision

AABB

Stability

70% planar support

Weight balance

Center of Gravity

Route

OSRM + TSP

Sea

MARNET + Dijkstra

Air

WGS84 Geodesic + Open-Meteo 250 hPa

Delay

HistGradientBoosting

Cost

HistGradientBoosting

Demand

2-layer PyTorch LSTM

Anomaly

Isolation Forest

Highway GNN

GraphSAGE

RL route

PPO + GNN

Computer vision

OpenCV Canny + Contours + Reference calibration

LLM

NVIDIA Nemotron through OpenRouter

Did we train the LLM?

NO

Did we train ML models?

YES

Genetic Algorithm?

NO

YOLO?

NO

XGBoost?

Listed but not actively used

Tests

10/10 — 100%

Biggest achievement

Synchronizing physical 3D loading depth with multi-stop route order.

Biggest limitation

Single-camera CV height estimation uses an aspect-ratio approximation.

These points are directly reflected in the dossier's one-page cheat sheet and implementation verification.

LogiLoad3D_Forensic_Technical_A...

61. THE ONE SENTENCE THAT EXPLAINS THE WHOLE PROJECT

If the examiner suddenly says:

"Explain your project in one sentence."

Say:

"LogiLoad 3D is a multi-modal logistics platform that determines how cargo should be routed, physically packed and weight-balanced across trucks, aircraft

and ships, while using machine learning, computer vision, telemetry and a grounded LLM to provide predictive and operational intelligence."

And if they say:

"What is unique about it?"

Say:

"Our key feature is the Hybrid Route-LIFO Synchronizer, which connects the optimized delivery sequence directly to the physical 3D loading depth, so cargo for the first delivery is kept closest to the door and cargo for later deliveries is loaded deeper."

LogiLoad3D_Forensic_Technical_A...

That is the story you should keep in your head tomorrow:

Problem → Input → Route → LIFO → 3D Packing → Physics/CoG → ML → AI → Visualization → Dispatch → GPS → Anomaly Detection → Result.

Once you understand that chain, the individual technologies become much easier to explain.

Sources

ChatGPT can make mistakes. Check important info.